

This program produces perfectly self-similar trees, as the `LEFT_ANGLE`, `LEFT_DECREASE`, `RIGHT_ANGLE`, and `RIGHT_DECREASE` variables are initially randomly chosen but stay constant for all the recursive calls. The `random.seed()` function sets a seed value for Python's random functions. The *random number seed value* causes the program to produce random-seeming numbers, but it uses the same sequence of random numbers for each branch of the tree. In other words, the same *seed* value reproduces the same *tree* each time you run the program. (I never apologize for my puns.)

To see this in action, enter the following into the Python interactive shell:

Python

```
>>> import random
>>> random.seed(42)
>>> [random.randint(0, 9) for i in range(20)]
[1, 0, 4, 3, 3, 2, 1, 8, 1, 9, 6, 0, 0, 1, 3, 3, 8, 9, 0, 8]
>>> [random.randint(0, 9) for i in range(20)]
[3, 8, 6, 3, 7, 9, 4, 0, 2, 6, 5, 4, 2, 3, 5, 1, 1, 6, 1, 5]
>>> random.seed(42)
>>> [random.randint(0, 9) for i in range(20)]
[1, 0, 4, 3, 3, 2, 1, 8, 1, 9, 6, 0, 0, 1, 3, 3, 8, 9, 0, 8]
```

In this example, we set the random seed to 42. When we generate 20 random integers, we get 1, 0, 4, 3, and so on. We can generate another 20 integers and continue to receive random integers. However, if we reset the seed to 42 and generate 20 random integers again, they'll be the same "random" integers as before.

If you'd like to create a more natural, less self-similar tree, replace the lines after the `# Record the position of the branch's end.` comment with the following lines. This generates new random angles and branch lengths for *every* recursive call, which is closer to the way trees grow in nature:

Python

```
# Record the position of the branch's end.
endPosition = turtle.position()
leftDirection = direction + random.randint(10, 30)
leftBranchLength = branchLength - random.randint(8, 15)
rightDirection = direction - random.randint(10, 30)
rightBranchLength = branchLength - random.randint(8, 15)
```
